Solution: Longest Continuous Subarray

Problem Statement

Given an array of integers `nums` and an integer `limit`, find the length of the longest contiguous `non-empty` subarray within `nums`, where the absolute difference between any two elements in this subarray does not exceed a specified `limit`.

Examples

1. **Example 1**:
    - **Input**: Array = [10, 1, 2, 4, 7], Limit = 5
    - **Expected Output**: 3
    - **Justification**: The longest subarray where the absolute difference between any two numbers is at most 5 is [1, 2, 4].
2. **Example 2**:
    - **Input**: Array = [4, 8, 5, 1, 7, 9], Limit = 3
    - **Expected Output**: 2
    - **Justification**: One possible longest subarray is [7, 9] where the difference between 7 and 9 is within the limit of 3.
3. **Example 3**:
    - **Input**: Array = [3, 3, 3, 3, 3], Limit = 0
    - **Expected Output**: 5
    - **Justification**: Since all elements are the same, the entire array is the longest subarray with differences of 0.

Solution

To solve this problem, we use a TreeMap to manage the elements of the current subarray as we iterate through the given array. The TreeMap helps in maintaining the order of elements and allows us to quickly access the smallest and largest elements.

As we traverse the array, we dynamically adjust the size of a sliding window. We add elements to the window and check if the absolute difference between the minimum and maximum elements within the window stays within the given limit. If the condition is violated, we shrink the window from the left side until the condition is met again. Throughout this process, we keep track of the length of the longest subarray found that satisfies the condition. The final result is the maximum length of such a subarray. This approach ensures that we examine each element of the array only once, making the solution efficient and effective for finding the longest subarray within the specified limit.

The algorithm can be described as follows:

1. **Initialization**: Start with two pointers (or indices) at the beginning of the array. Initialize an ordered set to keep track of the elements within the current window (subarray).
2. **Window Expansion**: Move the right pointer to expand the window until the difference between the maximum and minimum values in the ordered set exceeds the limit.
3. **Window Contraction**: Once the limit is exceeded, move the left pointer to shrink the window until the condition is satisfied again. During this process, update the ordered set accordingly.
4. **Result Update**: After each expansion, update the result with the maximum length of the window (subarray) found so far.

This approach ensures that at every stage, we maintain a subarray where the absolute difference condition is met, and we efficiently find the longest such subarray. The use of an ordered set is crucial for fast access to the minimum and maximum elements, which are the bottleneck operations in this problem.

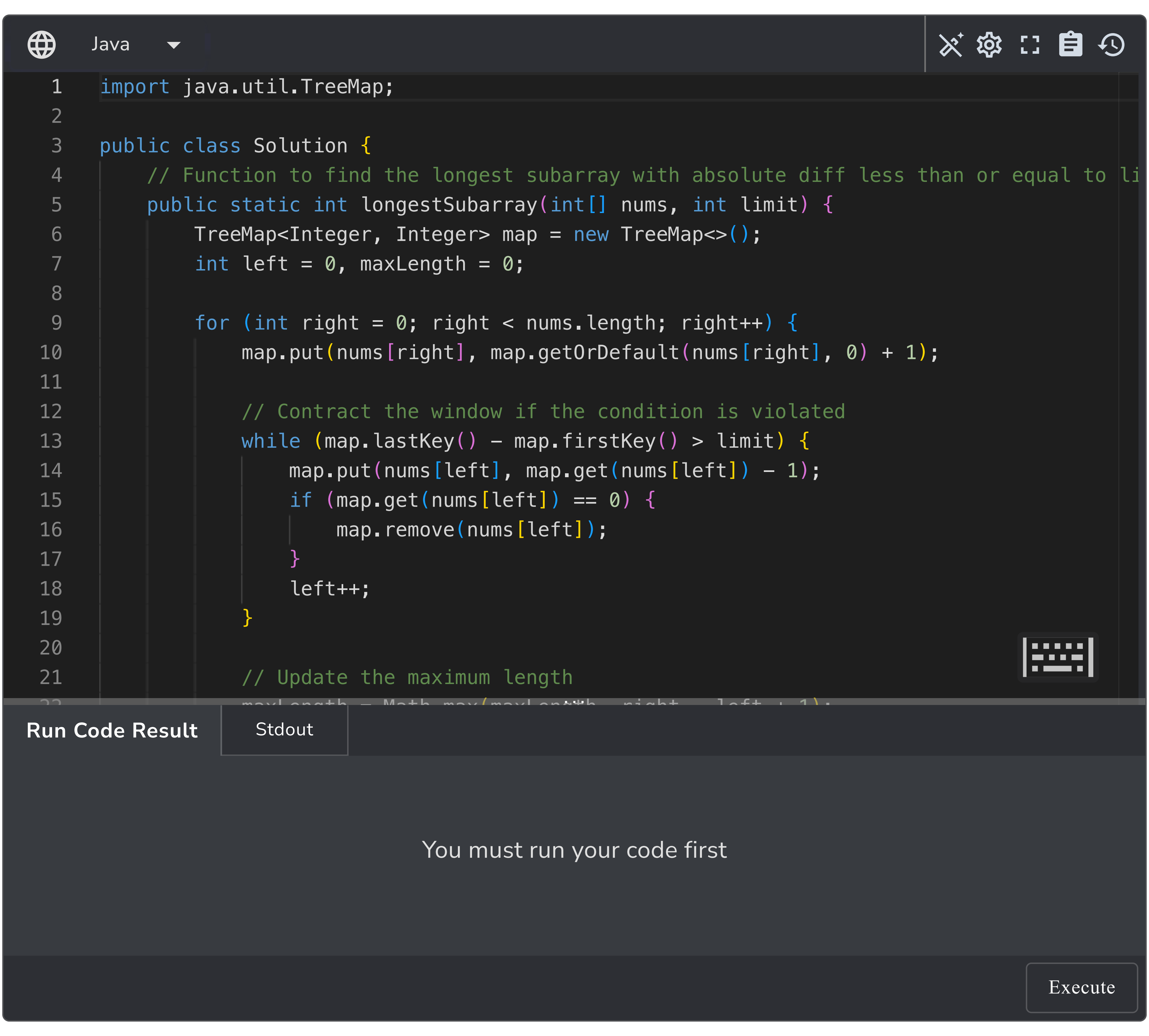
Algorithm Walkthrough

Let's walk through the first example step-by-step to understand how the algorithm works. The input for this example is an array `[10, 1, 2, 4, 7]` with a limit of `5`.

1. **Initialization**:
    - Start with two pointers, `left` and `right`, both at index `0`. Initialize an empty TreeMap (in Java) or equivalent ordered set/map.
2. **First Iteration** (**right = 0**):
    - Add `10` to the TreeMap.
    - Current TreeMap: `{10: 1}`
    - The condition (max - min <= limit) is satisfied (`10 - 10 = 0 <= 5`).
    - Current longest subarray length = `1` (`[10]`).
3. **Second Iteration** (**right = 1**):
    - Add `1` to the TreeMap.
    - Current TreeMap: `{1: 1, 10: 1}`
    - The condition is not satisfied (`10 - 1 = 9 > 5`).
    - Move `left` pointer right by one (remove `10` from TreeMap).
    - Current TreeMap after adjustment: `{1: 1}`
    - Current longest subarray length = `1` (reduced to `[1]`).
4. **Third Iteration** (**right = 2**):
    - Add `2` to the TreeMap.
    - Current TreeMap: `{1: 1, 2: 1}`
    - The condition is satisfied (`2 - 1 = 1 <= 5`).
    - Current longest subarray length = `2` (`[1, 2]`).
5. **Fourth Iteration** (**right = 3**):
    - Add `4` to the TreeMap.
    - Current TreeMap: `{1: 1, 2: 1, 4: 1}`
    - The condition is satisfied (`4 - 1 = 3 <= 5`).
    - Current longest subarray length = `3` (`[1, 2, 4]`).
6. **Fifth Iteration** (**right = 4**):
    - Add `7` to the TreeMap.
    - Current TreeMap: `{1: 1, 2: 1, 4: 1, 7: 1}`
    - The condition is satisfied (`7 - 1 = 6 > 5`).
    - Move `left` pointer right by one (remove `1` from TreeMap).
    - Current TreeMap after adjustment: `{2: 1, 4: 1, 7: 1}`
    - The condition is now satisfied (`7 - 2 = 5 <= 5`).
    - Current longest subarray length = `3` (`[2, 4, 7]`).

The algorithm ends here as we have reached the end of the array. The maximum length of the subarray found that satisfies the condition is `3`, which is the expected output for this example.

Code



```java
import java.util.TreeMap;

public class Solution {
    // Function to find the longest subarray with absolute diff less than or equal to li
    public static int longestSubarray(int[] nums, int limit) {
        TreeMap<Integer, Integer> map = new TreeMap<>();
        int left = 0, maxLength = 0;

        for (int right = 0; right < nums.length; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);

            // Contract the window if the condition is violated
            while (map.lastKey() - map.firstKey() > limit) {
                map.put(nums[left], map.get(nums[left]) - 1);
                if (map.get(nums[left]) == 0) {
                    map.remove(nums[left]);
                }
                left++;
            }

            // Update the maximum length
```

Complexity Analysis

- **Time Complexity**: The time complexity of this algorithm is O(N log N), where N is the number of elements in the array. This is due to the operations performed on the ordered set (or map in some implementations), like insertion, deletion, and finding the max/min elements, each taking O(log N) time.
- **Space Complexity**: The space complexity is O(N) due to the additional data structure (ordered set or map) used to store the elements of the current window.

← Previous
Longest Continuous Subarray (Medium)

Next →
Introduction To Multi-Threaded Pattern

☐ Mark as Completed